

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ



FACULTAD DE INGENIERÍA
ÁREA DE CIENCIAS DE LA COMPUTACIÓN

MARTINEZ SANCHEZ MAYRA ALIN
RODRIGUEZ DE LA TORRE AARON EMMANUEL
BRAGA DE LIMA BRENDA

INTERFACES GRÁFICAS CON APLICACIONES

M.I. Nuñez Varela Alberto Salvador

Reporte proyecto UNO

13/09/2026

SEMESTRE
2026-2027/I

Índice

1.Descripción general del proyecto

2.Arquitectura del proyecto

- 2.1 Capa de presentación (Windows Forms)
- 2.2 Capa de lógica del juego
- 2.3 Capa de acceso a datos (DAO)
- 2.4 Base de datos
- 2.5 API

3. Tabla de cuentas de GitHub

4. División de tareas

- 4.1 Braga de Lima Brenda
- 4.2 Martínez Sánchez Mayra Alin
- 4.3 Rodríguez de la Torre Aarón Emmanuel

5. Diagrama entidad-relación de la base de datos

6. Listado de commits

7. Cronograma de actividades

8. Resumen y gráfica de commits

9. Listado de ramas generadas

10. Capturas de la aplicación

11. Problemas encontrados y cómo los solucionaron

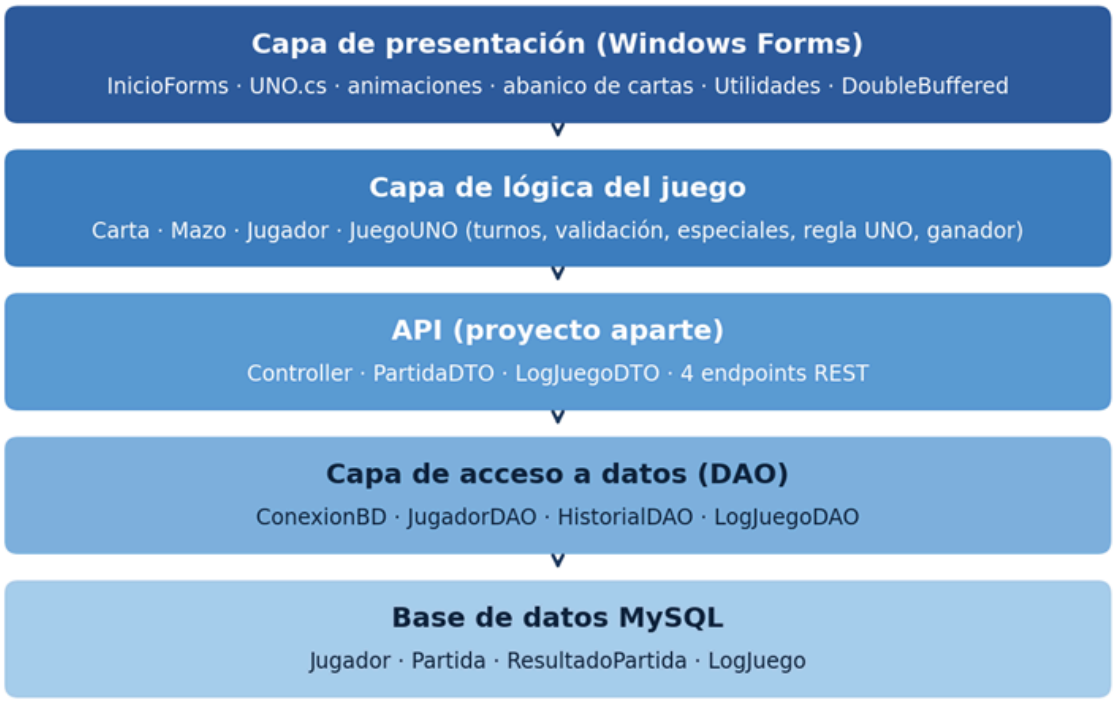
12. Conclusión personal

- Descripción general del proyecto

Este proyecto está enfocado en la realización de un juego UNO gráfico en C# ejecutado en una sola computadora, con conexión a APIs y base de datos para almacenar el registro de jugadores, el historial de partidas ganadas y perdidas y el log detallado de cada movimiento. Asimismo, incluye un reporte técnico con la descripción del sistema, división de tareas, diagrama entidad-relación, evidencias de Git/GitHub (ramas y commits) y la solución a los problemas encontrados

- Arquitectura del proyecto

El proyecto sigue una arquitectura en capas, desarrollada en C# con Windows Forms y conectada a una base de datos MySQL y a una API. Cada capa tiene una responsabilidad distinta.



1. Capa de presentación (Windows Forms)
Contiene el formulario de inicio (InicioForms) y el formulario principal del juego (UNO.cs). Muestra a los tres jugadores, sus cartas como botones, la carta actual, el turno, el botón UNO y el mensaje del ganador. Incluye las clases de animación (desplazamiento de cartas, abanico de cartas), la clase Utilidades y el uso de DoubleBuffered para evitar el parpadeo.

2. Capa de lógica del juego
Aquí están las reglas de UNO:

- Carta: color, valor y tipo.
- Mazo: barajado, robo y pila de descarte.
- Jugador: ID, nombre y mano de cartas.
- JuegoUNO: estado de la partida, jugador actual, dirección, turnos, validación de cartas, cartas especiales (Reversa, Salta, +2, Comodín y +4), regla UNO con su penalización, detección del ganador y lista de perdedores.

3. Capa de acceso a datos (DAO)

- ConexionBD: conexión con MySQL mediante el paquete MySql.Data y métodos base para ejecutar consultas.
- JugadorDAO: consulta de jugadores y sus estadísticas.
- HistorialDAO: guardado de partidas, ganadores, participantes y resultados.
- LogJuegoDAO: registro de cada movimiento (cartas jugadas y robadas, cambios de color, turnos, acciones especiales y fecha/hora).

4. Base de datos

Tiene las tablas Jugador, Partida, ResultadoPartida y LogJuego, con sus llaves primarias, foráneas y relaciones. Se insertan los tres jugadores iniciales.

5. API

Es un proyecto aparte con su clase controller y los DTO (PartidaDTO, LogJuegoDTO). Expone cuatro endpoints:

- GET /api/jugadores
- POST /api/partidas
- POST /api/partidas/resultados
- POST /api/log

El log se envía de forma asíncrona para no congelar la interfaz.

Flujo general: el jugador interactúa con el formulario, el formulario llama a JuegoUNO, que valida y aplica las reglas. Cada evento se registra mediante los DAO y la API en la base de datos, y al terminar la partida se guardan el ganador y los resultados.

- Tabla de cuentas de GitHub

INTEGRANTE	CUENTA DE GITHUB
Martinez Sanchez Mayra Alin	 alinsnhz Alin Sanchez
Rodriguez de la Torre Aaron Emmanuel	 Emmanuel1514
Braga de Lima Brenda	 brendabragaaaa

- # División de tareas

BRAGA DE LIMA BRENDA

implementar clase Carta

Crear la clase Carta que representa cada carta del UNO. Debe contener la información necesaria para identificar su color, valor y tipo de carta.

Tareas:
Crear Carta.cs
Agregar propiedad de color.
Agregar propiedad de valor.
Agregar propiedad de tipo.
Crear constructor.
Probar creación de cartas.

implementar cartas del UNO

Implementar la creación de las cartas que forman el mazo del UNO.

Tareas:
Crear cartas de números 0–9.
Crear cartas Reversa.
Crear cartas Salta.
Crear cartas +2.
Crear Comodines.
Crear Comodines +4.
Verificar que el mazo tenga la cantidad correcta de cartas.

implementar clase Mazo

Crear la clase encargada de administrar el mazo de cartas.

Tareas:

Crear Mazo.cs.
Crear lista de cartas.
Crear método para generar el mazo.
Crear método para barajar.
Crear método para sacar/robar una carta.
Controlar las cartas restantes.

Repartir cartas iniciales

Implementar el reparto inicial de cartas a los tres jugadores.

Tareas:
Repartir 7 cartas a cada jugador.
Verificar que no se repartan cartas inexistentes.
Verificar cantidad de cartas restantes.
Probar reparto para 3 jugadores.

Administrar pila de descarte

Implementar la pila donde se colocan las cartas jugadas y controlar la carta que se encuentra actualmente en juego.

Tareas:
Crear pila/lista de descarte.
Agregar cartas jugadas.
Obtener carta superior.
Controlar qué sucede cuando se termina el mazo.

MARTINEZ SANCHEZ MAYRA ALIN

implementar clase Jugador

Crear la clase que represente a cada uno de los tres jugadores.

Tareas:
Crear Jugador.cs.
Agregar ID.
Agregar nombre.
Agregar lista de cartas.
Agregar métodos para agregar cartas.
Agregar métodos para quitar cartas.
Probar los tres jugadores.

implementar clase JuegoUNO

Crear la clase principal encargada de controlar el estado de la partida.

Tareas:
Crear JuegoUNO.cs.
Agregar lista de jugadores.
Controlar al jugador actual.
Controlar la carta actual.
Controlar dirección del juego.
Crear método para iniciar partida.
Crear método para cambiar turnos.

implementar validación de cartas

Implementar las reglas que determinan si una carta puede ser jugada.

Tareas:
Permitir carta del mismo color.
Permitir carta del mismo número.
Permitir carta del mismo símbolo.
Permitir comodín.
Validar correctamente +4.
Mostrar/retornar si una carta es válida.

implementar cartas especiales

Implementar el comportamiento de las cartas especiales del UNO.

Tareas:
Implementar Reversa.
Implementar Salta.
Implementar +2.
Implementar Comodín.
Implementar +4.
Cambiar de dirección cuando corresponda.
Saltar turno cuando corresponda.
Hacer robar cartas cuando corresponda.

implementar sistema de turnos

Implementar el sistema de turnos para los tres jugadores.

Tareas:

Jugador 1 → Jugador 2 → Jugador 3.
Cambiar correctamente de jugador.
Permitir cambiar de dirección.
Controlar turnos después de cartas especiales.
Evitar que un jugador juegue fuera de su turno.

implementar acción de robar

Implementar la acción de robar cartas durante el turno.

Tareas:
Robar una carta cuando el jugador no pueda jugar.
Agregar la carta a su mano.
Comprobar si puede jugar la carta robada.
Registrar la acción para posteriormente enviarla al log.

implementar UNO y penalización

Implementar la lógica de la regla UNO, incluyendo la detección de jugadores con una sola carta, la declaración de UNO y la penalización de robar dos cartas cuando un jugador no declare UNO dentro de la oportunidad correspondiente.

Tareas:
Detectar cuando un jugador queda con una carta.
Indicar que debe declarar UNO.
Crear estado para saber si declaró UNO.
Validar la declaración de UNO.
Determinar cuándo termina la oportunidad de declarar UNO.
Detectar que un jugador no declaró UNO.
Aplicar penalización de 2 cartas.
Obtener las 2 cartas del mazo.
Agregar las cartas a la mano del jugador.
Registrar la penalización para el log.
Reiniciar el estado de UNO para el siguiente turno.

detectar ganador

Implementar la condición de victoria y finalización de partida.

Tareas:
Detectar jugadores sin cartas.
Finalizar la partida.
Identificar al ganador.
Identificar jugadores perdedores.
Preparar información para guardar resultados en BD

implementa cartas de interfaz

Diseñar y vincular la representación lógica de las cartas con sus elementos visuales correspondientes en el formulario de la interfaz del usuario

Tareas:
Asociar las propiedades de la clase Carta con imágenes e íconos correspondientes
Implementar métodos de renderizado para mostrar mano actual del jugador
Vincular eventos de clic en cartas con lógica de validación
Actualizar el estado visual de la mesa y la pila de descarte en cada jugada

cambios pendientes de realizar

Ajustes lógicos y optimizaciones de código requeridos para corregir inconsistencias durante las pruebas de integración del juego

Tareas:
Refactorizar métodos de cambio de turno para gestionar correctamente los efectos combinados de cartas (+2 y +4)
Corregir errores en la detección de fin de juego cuando se roba la última carta del mazo
Ajustar el tiempo de espera de los turnos de jugadores para sincronizar efectos visuales
Resolver conflictos de fusiones en ramas de Git

animaciones de cartas

Añadir retroalimentación visual al usuario durante las acciones principales de la partida

Tareas:
Crear clases y helpers para animar el desplazamiento de las cartas del mazo a la mano del jugador
Implementar animación de deslizamiento/transición al colocar una carta en la pila de descarte
Agregar efectos visuales al presionar el botón “UNO” o cuando se aplica una penalización

RODRIGUEZ DE LA TORRE AARON EMMANUEL

implementar interfaz principal

Crear la interfaz gráfica del juego en Windows Forms.

Tareas:
Crear formulario principal.
Mostrar los tres jugadores.
Mostrar las cartas de cada jugador.
Mostrar carta actual.

Mostrar turno actual.
Crear botones necesarios.
Crear mensaje de ganador.

mostrar cartas como botones

Implementar la representación visual de las cartas de los jugadores.

Tareas:
Mostrar las cartas de Jugador 1.
Mostrar las cartas de Jugador 2.
Mostrar las cartas de Jugador 3.
Permitir seleccionar una carta.
Actualizar la interfaz después de jugar.
Actualizar la interfaz después de robar.
Agregar botón UNO.
Habilitar/deshabilitar el botón según el estado de la partida.
Conectar el botón con el método de declaración de UNO.
Actualizar la interfaz después de declarar UNO.
Actualizar la interfaz si se aplican las 2 cartas de penalización.

crear base de datos

Crear la base de datos necesaria para almacenar la información solicitada por el proyecto.

Tareas:
Crear tabla Jugador.
Crear tabla Partida.
Crear tabla ResultadoPartida.
Crear tabla LogJuego.
Definir llaves primarias.
Definir llaves foráneas.
Crear relaciones.
Insertar los tres jugadores iniciales.

crear conexión con BD

Implementar la conexión entre el programa C# y la base de datos.

Tareas:
Crear ConexionBD.cs.
Configurar conexión.
Probar conexión.
Manejar errores de conexión.
Crear métodos básicos para ejecutar consultas.

DAO de jugadores

Crear las operaciones necesarias para consultar la información de los jugadores.

Tareas:
Crear JugadorDAO.cs.
Consultar jugadores.

Obtener jugador por ID.
Consultar estadísticas del jugador.

guardar historial de partidas

Implementar el almacenamiento de los resultados de cada partida.

Tareas:
Guardar partida.
Guardar ganador.
Guardar jugadores participantes.
Guardar resultado de cada jugador.
Registrar ganadas/perdidas.
Consultar historial.

implementar Log del juego

Registrar en la base de datos todos los movimientos realizados durante una partida.

Tareas:
Crear LogJuegoDAO.cs.
Registrar cartas jugadas.
Registrar cartas robadas.
Registrar cambios de color.
Registrar turnos.
Registrar acciones de cartas especiales.
Guardar fecha/hora.
Asociar cada movimiento con jugador y partida.

conectar interfaz con BD

Integrar la interfaz gráfica con las funciones de base de datos.

Tareas:
Guardar partida al finalizar.
Guardar resultados.
Guardar movimientos.
Mostrar información del jugador cuando sea necesario.
Verificar que los datos realmente se guarden.

animaciones de cartas

Optimizar el refresh de pantalla para evitar el parpadeo mediante técnicas de Double Buffering

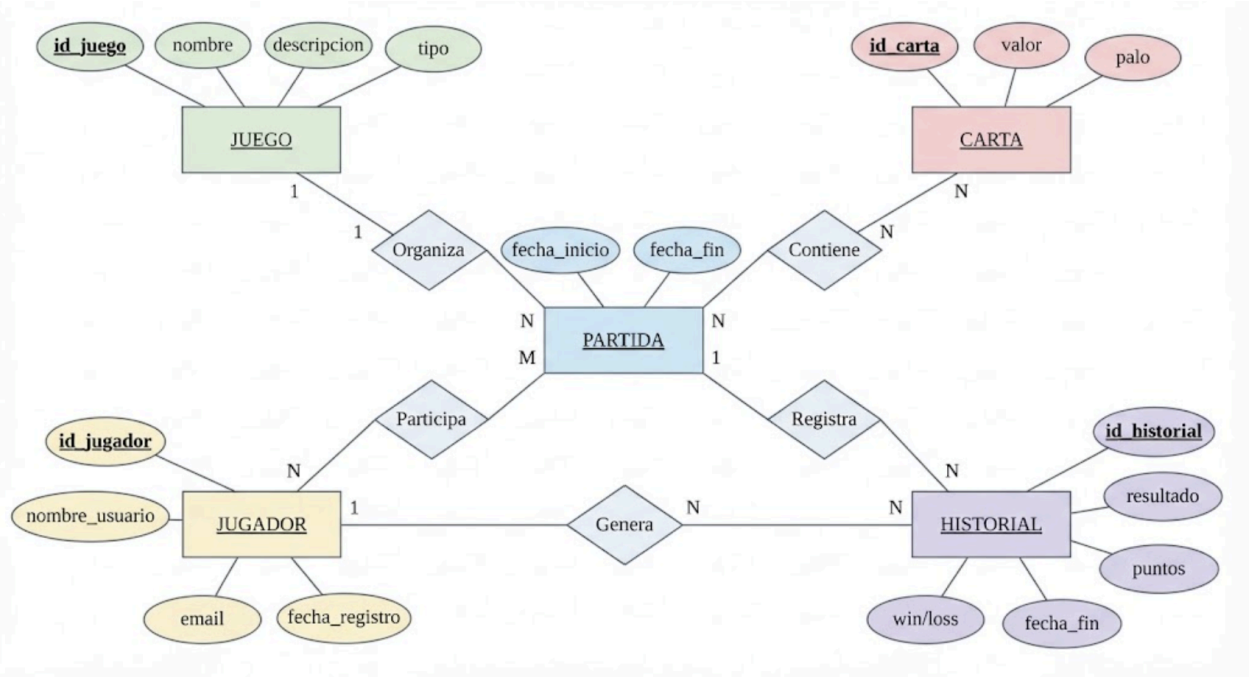
API

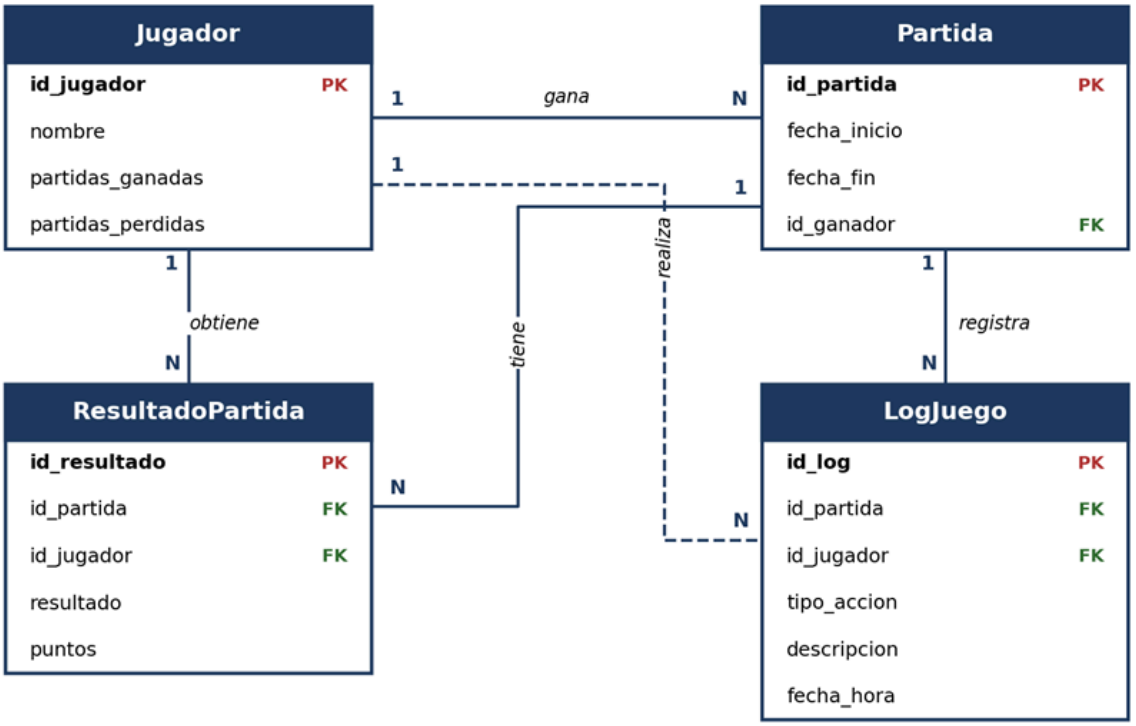
Implementar API al proyecto

Endpoints principales:
GET /api/jugadores: obtiene el catálogo de jugadores registrados y sus estadísticas generales
POST /api/partidas: registra el inicio de una nueva partida y almacena sus parámetros de configuración
POST /api/partidas/resultados: guarda los resultados finales, incluyendo el ganador y los puntajes de los perdedores
POST /api/log: registra de forma asíncrona cada evento o movimiento dentro del juego (cartas jugadas, robadas, penalizaciones y declaraciones de UNO)

La comunicación se realiza mediante objetos DTO (Data Transfer Objects) (PartidaDTO, LogJuegoDTO), garantizando la integridad de la información

- Diagrama entidad-relación de la base de datos





- Listado de commits

24/09/26 - 25/09/26

Commits on Sep 25, 2026

agrega nombre y lista de cartas de un jugador alinsnhz committed last week		8b66d54
agrega using de lista y declara variable ID a Jugador alinsnhz committed last week		68a86e8
crear clase Jugador alinsnhz committed last week		320bc2f
crear estructura proyecto UNO alinsnhz committed last week		de54feb

Commits on Sep 24, 2026

Initial commit Emmanuel1514 authored last week	Verified	c8c583c

Commits on Sep 25, 2026

agrega método para contabilizar cartas de un jugador alinsnhz committed last week		79e1c29
agrega método para quitar/eliminar cartas a un jugador alinsnhz committed last week		5be7b87
agrega método para añadir cartas a un jugador alinsnhz committed last week		cb89cb1
agrega constructor de Jugador alinsnhz committed last week		921fa29

26/09/26

Commits on Sep 26, 2026

crear método de validación de cartas alinsnhz committed 5 days ago		87f61b8
Mostrar cartas Emmanuel1514 committed 5 days ago		b954e20
Actualiza interfaz Emmanuel1514 committed 5 days ago		a141277
Panel del las cartas del jugador 3 Emmanuel1514 committed 5 days ago		56f15e0
cartas del jugador 2 Emmanuel1514 committed 5 days ago		3fde5f4
Se ponen las cartas como botones Aaron authored and Aaron committed 5 days ago		70bd8ee
15-crear-interfaz-principal Emmanuel1514 authored 5 days ago	Verified	7112f65
Merge branch '15-crear-interfaz-principal' of https://github.com/alinsnhz/proyecto-UNO into 15-crear-interfaz-principal Aaron authored and Aaron committed 5 days ago		f727e82
Se modifica fondo y se hace interfaz para mostrar las cartas y se muestra al player y su nombre Emmanuel1514 authored and Aaron committed 5 days ago		94df72e
Merge pull request #24 from alinsnhz/7-crear-clase-juegouno Emmanuel1514 authored 5 days ago	Verified	8d27b8e
agregar cambio de turno alinsnhz committed 5 days ago		e7ddc81
agrega método para iniciar partida alinsnhz committed 5 days ago		ab68605
agrega constructor para inicializar el estado del juego alinsnhz committed 5 days ago		3b53923
agrega control de la direccion de la jugada alinsnhz committed 5 days ago		6fe29b6

agregar control de carta actual
alinsnhz committed 5 days ago — 6e8a61e

agrega control de jugador actual
alinsnhz committed 5 days ago — 47865da

agrega clase jugadorUNO
alinsnhz committed 5 days ago — 2631385

Merge branch '7-crear-clase-juegouno' of https://github.com/alinsnhz/proyecto-UNO into 7-crear-clase-juegouno
alinsnhz committed 5 days ago — 3f0c981

agregar clase juegoUNO y lista de jugadores
alinsnhz committed 5 days ago — 654ea52

Merge pull request #23 from alinsnhz/6-crear-clase-jugador
Emmanuel1514 authored 5 days ago — Verified — 114ab64

27/09/26

Commits on Sep 27, 2026

Merge pull request #27 from alinsnhz/9-implementar-cartas-especiales
alinsnhz authored 4 days ago — Verified — 35c43fe

Merge branch '9-implementar-cartas-especiales' of https://github.com/alinsnhz/proyecto-UNO into 9-implementar-cartas-especiales
alinsnhz committed 4 days ago — 0f7a30a

agrega cambios de rama
alinsnhz committed 4 days ago — aa9c18a

Merge branch 'main' into 9-implementar-cartas-especiales
alinsnhz authored 4 days ago — Verified — e593998

Merge pull request #26 from alinsnhz/8-implementar-validación-de-cartas
alinsnhz authored 4 days ago — Verified — d1f0800

agrega método para regla +4
alinsnhz committed 4 days ago — 4c185b3

agrega método para implementar comodín de color
alinsnhz committed 4 days ago — 7f0f8a5

agrega método para la regla +2
alinsnhz committed 4 days ago — 24c673e

agrega método para la regla salto de turno
alinsnhz committed 4 days ago — b66f3e8

agrega método para aplicar reversa
alinsnhz committed 4 days ago — 4b5792a

pendiente de modificar
alinsnhz committed 4 days ago — a914611

agrega método para obtener jugador actual
alinsnhz committed 4 days ago — 8c5561c

resolución de conflictos
alinsnhz committed 4 days ago — 1e5d8aa

actualiza rama
alinsnhz committed 4 days ago — 4e73a25

28/09/26

Commits on Sep 28, 2026

Crear clase Carta brendabragaaaa committed 3 days ago		e75503f
Merge pull request #32 from alinsnhz/12-implementar-uno alinsnhz authored 3 days ago	Verified	95c345f
Merge remote-tracking branch 'origin/main' into 12-implementar-uno alinsnhz committed 3 days ago		fc0f8d3
agrega método para procesar regla completa de UNO alinsnhz committed 3 days ago		3447394
agrega método para reiniciar estado de UNO alinsnhz committed 3 days ago		e3e6157
17-crear-base-de-datos Emmanuel1514 authored 3 days ago	Verified	a4ef2e9
Se crea la Base de Datos Emmanuel1514 committed 3 days ago		ca7ad30
agrega penalización de dos cartas por no declarar UNO alinsnhz committed 3 days ago		108b0c5
agrega método para penalizar falta de declaración de UNO alinsnhz committed 3 days ago		2cd0695
agrega control de oportunidad de declarar UNO alinsnhz committed 3 days ago		41865fb
agrega implementación de declaración de UNO alinsnhz committed 3 days ago		e0e1ca4
agrega método para indicar cuando el jugador debe declarar UNO alinsnhz committed 3 days ago		d5d0bc0
agrega método para detectar jugador con una carta alinsnhz committed 3 days ago		0496c4a
agrega estado de declaración de UNO alinsnhz committed 3 days ago		c6a56fb

16-mostrar-cartas-como-botones Emmanuel1514 authored 3 days ago	Verified	9915829
Se hacen Botones de uno y Robar para los otros jugadores y ademas se añade Mensaje de UNO al momento que un jugador se queda con 1 carta Emmanuel1514 committed 3 days ago		2ca3e98
Se ajustan los tamaños de las cartas de los jugadores 2 y 3 Emmanuel1514 committed 3 days ago		8666e23
Se ajustan los Botones de las cartas y de los jugadores Emmanuel1514 committed 3 days ago		7c6405b

Commits on Sep 28, 2026

Merge branch '1-crear-clase-carta' brendabragaaaa committed 3 days ago		f101319

30/09/26

Commits on Sep 30, 2026

Merge pull request #36 from alinsnhz/13-detectar-ganador alinsnhz authored 15 hours ago	Verified	12d0433
agrega método para resultado en guardar para BD alinsnhz committed 15 hours ago		3a2a6cb
agrega lista de jugadores perdedores alinsnhz committed 15 hours ago		e184621
modifica para impedir acciones después de finalizar partida alinsnhz committed 15 hours ago		38e5396
agrega método para finalizar la partida cuando hay un ganador alinsnhz committed 15 hours ago		916eae0
agrega para guardar jugador ganador alinsnhz committed 15 hours ago		e9661ea
agrega método para detectar jugador sin cartas alinsnhz committed 15 hours ago		0a7dcdb
agrega estado de partida terminada alinsnhz committed 15 hours ago		10acecd
20-guardar-historial-de-partidas Emmanuel1514 authored 18 hours ago	Verified	a22fd8d
Se crea Metodo para Consultar Historial en la BD Emmanuel1514 committed 18 hours ago		ebdf469
Se agrega metodo para Registrar las partidad ganadas y perdidas Emmanuel1514 committed 18 hours ago		3cc9744
Se añade metodo para Guardar el Resultado de l aPartida Emmanuel1514 committed 18 hours ago		43a63c4
Se Añade metodo para Guardar los participantes Emmanuel1514 committed 18 hours ago		222de3e
Se añade Metodo para guardar al Ganador Emmanuel1514 committed 18 hours ago		2ab0f2f
se agrega metodo para Guardar Partida Emmanuel1514 committed 18 hours ago		49fffaa
Se crea Clase HistorialDAO Emmanuel1514 committed yesterday		ab017e6
19-dao-de-jugadores Emmanuel1514 authored yesterday	Verified	d4e32fc
. Emmanuel1514 committed yesterday		99a8ce5
Se crea MEtodo para consultar estadisticas del Jugador Emmanuel1514 committed yesterday		1e0d65c
Se crea Metodo para obtener jugadores por ID Emmanuel1514 committed yesterday		752463a
Se crea Metodo para consultar a los Jugadores Emmanuel1514 committed yesterday		e5ae701
se crea metodo para Obtener O Crear al jugador Emmanuel1514 committed yesterday		3a6f658
Se crea metodo para obtener al jugador por el nombre Emmanuel1514 committed yesterday		1cc2755
se crea Metodo para Eliminar jugador Emmanuel1514 committed yesterday		1d9f3d2
se crea metodo para actualizar la BD Emmanuel1514 committed yesterday		300130b
se crea metodo para crear al jugaor Emmanuel1514 committed yesterday		2599456
se crea clase JugadorDAO Emmanuel1514 committed yesterday		b016a92
18-crear-conexión-con-BD Emmanuel1514 authored yesterday	Verified	df04ff9

se crean métodos básicos para ejecutar consultas
Emmanuel1514 committed yesterday — 27a102a

Conexion con la BD
Emmanuel1514 committed yesterday — 6f07e52

Se hace coneccion con la BD y se instalan paquetes de MYSql.Data
Emmanuel1514 committed yesterday — 62a9a82

Se crea clase Conexion con BD
Emmanuel1514 committed yesterday — 116a86f

Se cambia de ubicacion la BD a la carpeta winfosforms1/windowsform1
Emmanuel1514 committed yesterday — ddbc25c

01/10/26

Commits on Oct 1, 2026

Merge pull request #37 from alinsnhz/21-implementar-log-del-juego
Emmanuel1514 authored last week — Verified — 444eacf

Se modifica BD para el log
Emmanuel1514 committed last week — 180eccd

Se añade metodo para ostrar LOG de partida
Emmanuel1514 committed last week — 21333d9

Se crea metodo para Registrar Acciones Especiales
Emmanuel1514 committed last week — d7c1391

Se crea metodo para registrar turnos
Emmanuel1514 committed last week — f9aa929

Se crea metodo para registrar cabos de color
Emmanuel1514 committed last week — 45661ac

Se crea metodo para Registrar cartas robadas
Emmanuel1514 committed last week — 96eb4a5

Se crea Metodo para regustrar las cartas jugadas
Emmanuel1514 committed last week — d1f1474

Se crea metodo para guardar fecha y hora
Emmanuel1514 committed last week — d38e78f

Delete proyectoUNO directory
alinsnhz authored last week — Verified — d262331

02/10/26

Commits on Oct 2, 2026

Implementar creación del mazo UNO y ajustar Carta
brendabragaaaa committed last week — db8d46f

05/10/26

Commits on Oct 5, 2026

Merge pull request #40 from alinsnhz/39-implementa-cartas-de-interfaz
alinsnhz authored 2 days ago — Verified — 59fd0b3

Eliminar .vs/slnx.sqlite para resolver conflictos
alinsnhz committed 2 days ago — 6ba6144

modifica UNO.cs para implementar ruta de imágenes
alinsnhz committed 2 days ago — 5acdfd5

agrega cartas comodines
alinsnhz committed 2 days ago — cf8fb01

agrega cartas azul
alinsnhz committed 2 days ago — c78ec67

agrega cartas verde
alinsnhz committed 2 days ago — 0c8cbb6

agrega cartas amarillas y comodines de reversa y cancelar turno
alinsnhz committed 3 days ago — 1208a2f

agrega imagenes cartas rojas
alinsnhz committed 3 days ago — 86ad100

agrega carpeta para imagenes
alinsnhz committed 3 days ago — d557d29

Implementar reparto inicial de cartas
brendabragaaaa committed 3 days ago — 7d39366

Crear clase Mazo para administrar el mazo
brendabragaaaa committed 3 days ago — 4322134

06/10/26

Commits on Oct 6, 2026

Merge pull request #42 from alinsnhz/22-conectar-interfaz-con-bd Emmanuel1514 authored 2 days ago	Verified	44ac931
Merge branch 'main' into 22-conectar-interfaz-con-bd Emmanuel1514 authored 2 days ago	Verified	274baa1
Se conecta la interfaz con la bd Emmanuel1514 committed 2 days ago		df67918
se modifica LOGjuegoDAO Emmanuel1514 committed 2 days ago		f436efd
cambios ?? Emmanuel1514 committed 2 days ago		97c9c2b

Commits on Oct 6, 2026

modifica sql alinsnhz committed 2 days ago		ba1664c
modifica clase UNO (otra vez) alinsnhz committed 2 days ago		df443dc
modifica clase UNO alinsnhz committed 2 days ago		e4c3589
cambios de clases alinsnhz committed 2 days ago		80aab2d
staged alinsnhz committed 2 days ago		242f177
resuelve conflictos alinsnhz committed 2 days ago		d041e6d
Merge pull request #45 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	c77e6d4
Merge branch 'main' into 43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	da1edc0
modifica clase UNO y LogJuegoDAO alinsnhz committed 2 days ago		911c291
modifica clase juegoUNO alinsnhz committed 2 days ago		93628f4
cambia ruta de cartas alinsnhz committed 2 days ago		b1ca50c
Merge pull request #44 from alinsnhz/38-api Emmanuel1514 authored 2 days ago	Verified	8a199ee
cambios alinsnhz committed 2 days ago		c67b4aa
se agrega Api a la clase UNO Emmanuel1514 committed 2 days ago		4c2289b
cambia ruta de imagen alinsnhz committed 2 days ago		edac364
Merge branch 'main' of https://github.com/alinsnhz/proyecto-UNO alinsnhz committed 2 days ago		6639320
agrega cambios alinsnhz committed 2 days ago		f5f5b2f
Merge branch '5-adiministrar-pila-descarte' brendabragaaaa committed 2 days ago		c9c8a9e
Implementar pila de descarte y carta actual brendabragaaaa committed 2 days ago		bb0b4e9
se crea API Emmanuel1514 committed 2 days ago		e1e9d48
se crea Clase controller Emmanuel1514 committed 2 days ago		705e98a
se crea Proyecto API y se crea clase PartidaDtos (los objetos que transfieren datos) Emmanuel1514 committed 2 days ago		c07fe44

07/10/26

Commits on Oct 7, 2026

agrega clases para animaciones de cartas alinsnhz committed 18 hours ago		cbb48ed
agrega forms de inicio alinsnhz committed 19 hours ago		b049038
agrega clase para abanico de Cartas alinsnhz committed 19 hours ago		f8338bb
agrega cambios de clases alinsnhz committed 19 hours ago		222eadd
agrega clase Utilidades alinsnhz committed 19 hours ago		7f01aea
Merge pull request #47 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored yesterday	Verified	445b35f
agrega imagenes para iconos alinsnhz committed yesterday		80280c7
Merge pull request #46 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored yesterday	Verified	03f8fd9
cambios alinsnhz committed yesterday		291b348
agrega cambios a clase Jugador y conexion alinsnhz committed yesterday		c4cf493

08/10/26

Commits on Oct 8, 2026

modifica conflictos en clase UNO alinsnhz committed 2 days ago		0bcde0e
agrega cambios alinsnhz committed 2 days ago		9f68006
agrega iconos y cambio de clases alinsnhz committed 2 days ago		640dc12
Merge pull request #49 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	baf637b
resuelve conflictos alinsnhz committed 2 days ago		acef0bf
agrega random y verifica reglas del juego alinsnhz committed 2 days ago		c51087a
agrega cambio de iconos random alinsnhz committed 2 days ago		707c7a0
Merge pull request #48 from alinsnhz/43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	829421f
Merge branch 'main' into 43-cambios-pendientes-a-realizar alinsnhz authored 2 days ago	Verified	0bb738a
agrega ventana de inicio alinsnhz committed 2 days ago		a677066

09/10/26

Commits on Oct 9, 2026

Merge pull request #50 from alinsnhz/43-cambios-pendientes-a-realizar Emmanuel1514 authored yesterday	Verified	0e1e09b
Merge branch 'main' into 43-cambios-pendientes-a-realizar Emmanuel1514 authored yesterday	Verified	8fdf0db
. Emmanuel1514 committed yesterday		268813c
Se Documenta InicioForms Emmanuel1514 committed yesterday		40ca1d6
Se hace que al mostrar el mensaje del ganador se salga a la ventana principal y se añade icono Emmanuel1514 committed yesterday		24bc378
Se añade Doble bufer para evitar parpadeo Emmanuel1514 committed yesterday		f12c7d3

10/10/26

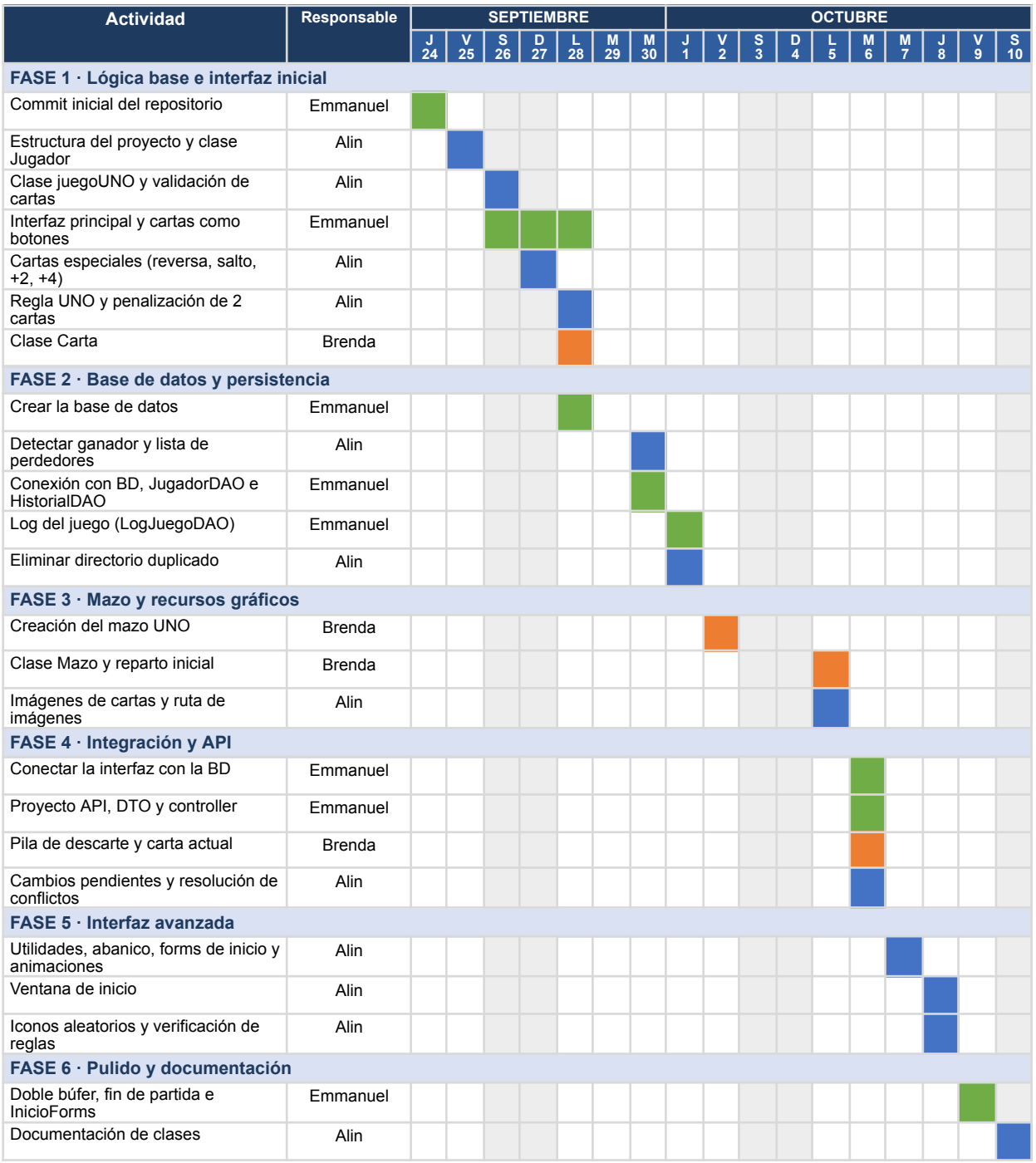
Commits on Oct 10, 2026

agrega documentación de clases alinsnhz committed 5 minutes ago	177fbde

- Cronograma de actividades

El cronograma se elaboró a partir del listado de commits del repositorio, del 24 de septiembre al 10 de octubre de 2026. Cada barra indica los días en que se trabajó la actividad y su color corresponde al responsable. Las actividades del 30 de septiembre se agruparon en ese día porque los commits se registraron en horas cercanas.

Responsables: Alin · Emmanuel · Brenda · Fin de semana



Todas las actividades se encuentran completadas al 10 de octubre de 2026.

- Resumen

RESUMEN POR RESPONSABLE

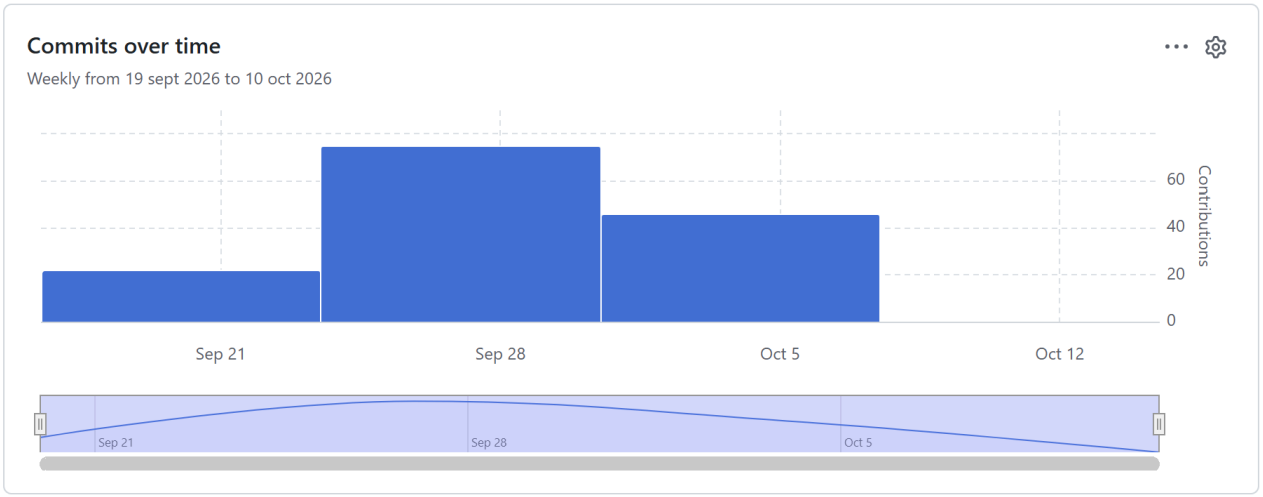
Responsable	Cuenta GitHub	Actividades	Días de trabajo	Commits (a main)
Alin	alinsnhz	13	13	87
Emmanuel	Emmanuel1514	12	14	51
Brenda	brendabragaaaa	4	4	5
Total		**29**	**31**	**143**

- Gráfica de commits

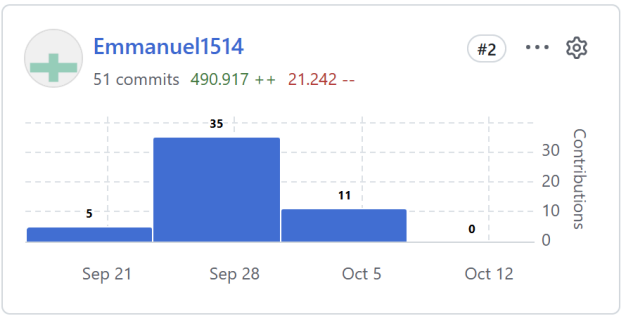
Contributors

Contributions per week to main, excluding merge commits

Period: All | Contributions: Commits









- Listado de ramas generadas

Branch	Updated	Check status	Behind	Ahead	Pull request
main	7 minutes ago		Default		
43-cambios-pendientes-a-realizar	yesterday		2	0	#50
38-api	3 days ago		45	1	
5-adiministrar-pila-descarte	4 days ago		65	0	
22-conectar-interfaz-con-bd	4 days ago		50	0	#42
39-implementa-cartas-de-interfaz	5 days ago		55	0	#41
4-repartir-cartas-iniciales	5 days ago		66	0	
3-crear-clase-mazo	5 days ago		67	0	
2-crear-cartas-uno	last week		68	0	
21-implementar-log-del-juego	last week		79	0	#37
13-detectar-ganador	last week		80	0	
14-pruebas-de-todas-las-reglas	last week		80	0	
20-guardar-historial-de-partidas	last week		89	0	#35
19-dao-de-jugadores	last week		97	0	#34
18-crear-conexión-con-bd	last week		108	0	
12-implementar-uno	2 weeks ago		113	0	
1-crear-clase-carta	2 weeks ago		125	0	
17-crear-base-de-datos	2 weeks ago		127	0	#31
11-implementar-acción-de-robar	2 weeks ago		128	0	
16-mostrar-cartas-como-botones	2 weeks ago		158	0	#30

Branch	Updated	Check status	Behind	Ahead	Pull request
10-implementar-sistema-de-turnos	2 weeks ago		145	0	
9-implementar-cartas-especiales	2 weeks ago		154	0	
8-implementar-validación-de-cartas	2 weeks ago		164	0	
7-crear-clase-juegouno	2 weeks ago		166	0	
15-crear-interfaz-principal	2 weeks ago		167	0	#25
6-crear-clase-jugador	2 weeks ago		180	0	#23

Capturas de la aplicación

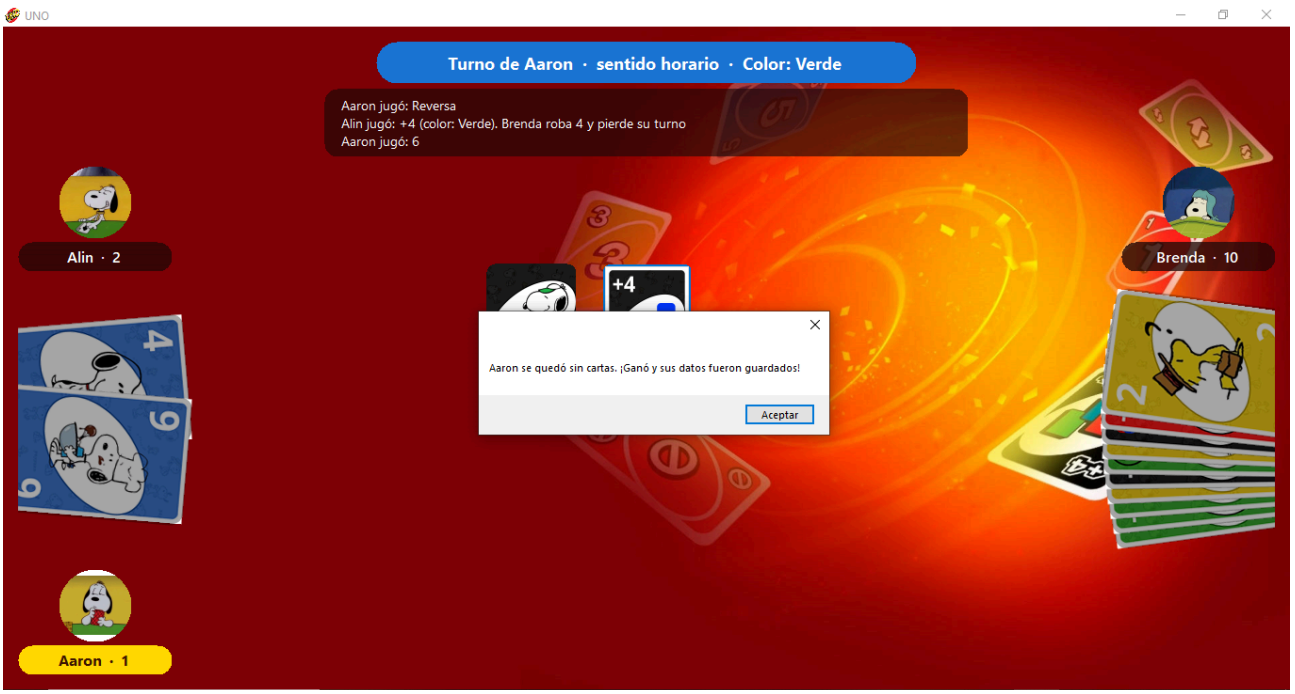
En esta sección se muestran las pantallas principales del juego en funcionamiento.



Ventana de inicio.



Partida en curso con los tres jugadores.



Mensaje del ganador.

- Problemas encontrados y cómo los solucionaron

1. Conflictos de fusión (*Merge Conflicts*) en Git

- **Problema:** Al trabajar de forma paralela en las clases lógicas principales (UNO.cs, JuegoUNO.cs) y la capa de datos, se generaron conflictos al hacer *Merge Pull Request* en las ramas de integración.
- **Solución:** Se establecieron reglas de sincronización previa (git pull origin main) antes de subir *commits*, y se resolvieron manualmente los conflictos revisando el código línea por línea en VS Code/Visual Studio antes de aprobar las solicitudes de extracción

2. Parpadeo visual (*Flicker*) y renderizado lento de cartas en la Interfaz Gráfica

- **Problema:** Al actualizar la mano del jugador o ejecutar animaciones de movimiento de cartas, la pantalla mostraba un parpadeo constante (*flicker*) debido al redibujado de los controles nativos de Windows Forms.
- **Solución:** Se habilitó la propiedad DoubleBuffered = true en el formulario principal y en los contenedores de las cartas, además de optimizar la carga de imágenes usando recursos almacenados en memoria local en lugar de cargarlos directamente desde el disco en cada turno.

3. Manejo de estados especiales y penalización de la regla "UNO"

- **Problema:** El juego no detectaba con precisión el momento exacto en que un jugador omitía presionar el botón "UNO" antes de pasar el turno, provocando que la penalización de 2 cartas no se aplicara a tiempo o se aplicara al jugador equivocado.

- **Solución:** Se implementó una bandera de estado (*boolean flag*) dentro del controlador de turnos (JuegoUNO.cs) que evalúa si el jugador tiene 1 sola carta al momento de finalizar su jugada. Si la declaración de UNO no fue activada en ese lapso, el sistema aplica automáticamente el método de penalización para robar dos cartas e incluye el registro en el log.

4. Persistencia asíncrona del Log de Juego en la Base de Datos

- **Problema:** Guardar cada movimiento (cartas robadas, jugadas, cambios de color) de manera síncrona producía pequeñas pausas o congelamientos en la interfaz de usuario durante la partida.
- **Solución:** Se reestructuraron las consultas del DAO (LogJuegoDAO) y los llamados a la API para que las operaciones de inserción en la base de datos SQLite/MySQL se ejecuten de fondo en segundo plano (*Background Worker / Async Tasks*), manteniendo la interfaz fluida.

- Conclusión personal

Martinez Sanchez Mayra Alin

Este proyecto me permitió aplicar lo visto en la materia: crear una interfaz gráfica en C# y conectarla con una base de datos y una API, organizando todo en capas. Me encargué sobre todo de la lógica del juego (turnos, validación de cartas, cartas especiales, regla UNO y detección del ganador) y de integrar con la interfaz. Lo más difícil fue lograr que todo funcionara junto, en especial los efectos combinados de +2 y +4 y la penalización por no declarar UNO.

También aprendí a trabajar en equipo con Git y GitHub: ramas por tarea, pull requests y resolución de conflictos. Si lo repitiera, haría las pruebas de integración antes y repartiría el trabajo de forma más equilibrada. Aun así, el resultado cumple con lo pedido: un juego funcional con historial de partidas y log de movimientos.

Rodriguez de la Torre Aaron Emmanuel

Este proyecto me permitió aplicar lo visto en la materia al construir un juego de UNO con interfaz gráfica en C#, conectado a una base de datos MySQL y a una API. Me encargué de la interfaz principal en Windows Forms, de mostrar las cartas de los tres jugadores como botones, de la base de datos, la conexión y los DAO (jugadores, historial y log del juego). También hice el proyecto API con sus DTO y el doble búfer para evitar el parpadeo.

Lo más difícil fue lograr que el registro del log no congelara la interfaz y que la pantalla se actualizará sin parpadeos. Resolverlo me enseñó a separar bien las capas del sistema y a pensar en el rendimiento desde el diseño.